

"synchronized locks the **method**, right?"

Wrong.

90% of candidates get this wrong in interviews.

It doesn't lock the method. It doesn't lock the class.

It locks the **object**. And everything changes once you understand that.



The Mutex Misconception

What synchronized ACTUALLY locks



ReentrantLock — 4 Superpowers

Why synchronized isn't enough



Semaphore, Latch, Barrier

When to use which — visual decision map



5 Problems Asked in MY DSA Rounds

Matrix, Producer-Consumer, Dining Philosophers...

Swipe → to **unlearn the wrong & learn it right**



Amaan Nirban

Java • Concurrency • DSA

01 / 07



Save

Follow +

CONCEPT 1

synchronized locks the ???

Let's kill the misconception once and for all.

✗ What most people think

- "It locks the **method**"
- "It locks the **class**"
- "It locks the **code block**"

All three are **WRONG**.

✓ What it actually does

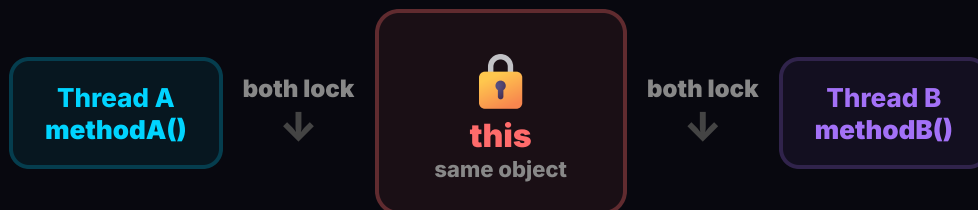
synchronized acquires a

mutex lock on an OBJECT

- **Method-level:** locks **this** (the instance)
- **Static method:** locks **ClassName.class**
- **Block-level:** locks **whatever object you pass**

The **OBJECT** is the lock. Always.

SAME OBJECT = SAME LOCK = BLOCKED



⚠ Thread B is **BLOCKED** even though it calls a **DIFFERENT** method!

💡 Fix: Use different lock objects

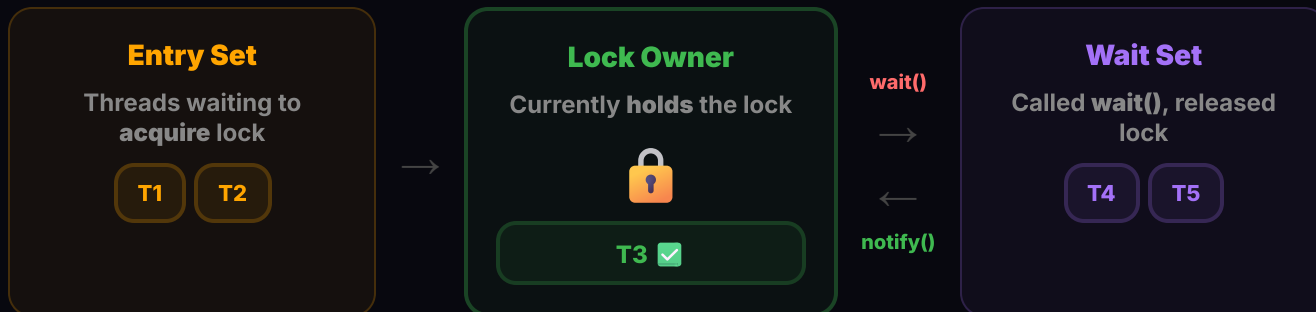
```
private final Object lockA = new Object(); private final Object lockB = new
Object(); void methodA() { synchronized(lockA) { ... } } // independent! void
methodB() { synchronized(lockB) { ... } } // runs in parallel
```

CONCEPT 2

wait() / notify() — The Monitor

Every Java object is a **monitor** with 3 parts. This is how threads talk to each other.

JAVA MONITOR MODEL



Rule 1

`wait()` / `notify()` **MUST** be called inside **synchronized**

Rule 2

Always use **while(!condition)** not **if** — spurious wakeups!

Rule 3

`wait()` **releases the lock**. Thread re-acquires it on wakeup.

```
synchronized(lock) { while(!condition) { // while, NOT if! lock.wait(); // releases lock + sleeps } // condition is true, do work lock.notifyAll(); // wake up ALL waiters }
```



Amaan Nirban

Java • Concurrency • DSA

Save

Follow +

CONCEPT 3

ReentrantLock — 4 Superpowers

Everything **synchronized** can't do. This is what interviewers want you to know.



1. tryLock(timeout)

Try to acquire for **N seconds**.

Fail? Do something else.

synchronized blocks FOREVER.



2. Fairness

new ReentrantLock(true)

Longest-waiting thread goes first.

synchronized has NO fairness.



3. Multiple Conditions

notFull.await() / notEmpty.signal()

Separate wait-sets per condition.

synchronized has only ONE wait-set.



4. lockInterruptibly()

Waiting thread can be **interrupted**.

Break out of a deadlock.

synchronized can't be interrupted.

TEMPLATE — ALWAYS USE TRY-FINALLY

```
Lock lock = new ReentrantLock(); lock.lock(); try { // critical section } finally  
{ lock.unlock(); // ALWAYS in finally! Else = permanent lock }
```

⚠ **Forget the finally block = lock is held forever. #1 mistake in interviews.**



Amaan Nirban

Java • Concurrency • DSA

🔥 Save

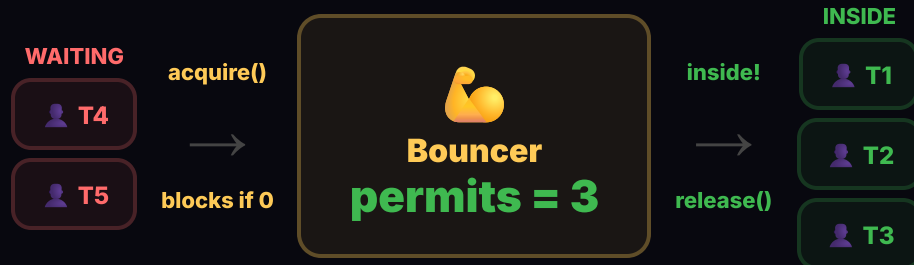
Follow +

CONCEPT 4

Semaphore — Permit-Based Control

Not a lock. It's a **bouncer**. Only N threads allowed inside at once.

NIGHTCLUB ANALOGY — SEMAPHORE(3)



🔒 **Mutex / Lock**

- **1 thread** at a time
- Has **ownership** — only owner unlocks
- **Reentrant** — same thread can re-lock

🚧 **Semaphore**

- **N threads** at a time
- **No ownership** — anyone can release
- **Not reentrant** — acquire again = -1 permit

💡 **Semaphore(1) ≠ Mutex! — No ownership, no reentrancy. Interview trap!**



Amaan Nirban

Java • Concurrency • DSA

🔖 Save

Follow +

CONCEPT 5

CountDownLatch vs CyclicBarrier

Both make threads wait for each other. But they work **completely differently**.

CountDownLatch(3)

"I'll wait until **3 events** happen"

Worker 1 → countDown() **3→2**

Worker 2 → countDown() **2→1**

Worker 3 → countDown() **1→0** ✓

Main thread: await()
released when count hits 0

CyclicBarrier(3)

"**ALL 3** of us wait for each other"

Thread 1 → await() ⌚ waiting...

Thread 2 → await() ⌚ waiting...

Thread 3 → await() ✨ **ALL GO!**

All threads rendezvous
resets & can be used again!

Feature	CountDownLatch	CyclicBarrier
Who waits?	1 thread awaits, N count down	All N wait for each other
Reusable?	No — one shot	Yes — auto-resets
Action on trip?	No	Yes — Runnable in constructor
Use case	Service startup, test sync	Phased computation, gaming

💡 **Latch** = "gate opens once" • **Barrier** = "everyone arrives, gate opens, repeat"

Now try **these problems**.

These were asked to me in actual interviews. Can you solve them?

1

Print M×N Matrix Row-by-Row Using N Threads

Each thread owns one column. Strict ordering — row 0 before row 1.

AMAZON

GOLDMAN SACHS

MORGAN STANLEY

2

Producer-Consumer Bounded Buffer

Implement thread-safe BlockingQueue. Solve it 3 ways.

MORGAN STANLEY

GOOGLE

UBER

3

Dining Philosophers — No Deadlock, No Starvation

5 philosophers, 5 forks. 3 different approaches expected.

MICROSOFT

AMAZON

LC 1226

4

Print FooBar Alternately + Zero Even Odd + H2O

Thread ordering family — all use the same Semaphore pattern.

LC 1115

LC 1116

LC 1117

LC 1114

Want **solutions + template code**?

I wrote a full deep-dive guide with visuals, code templates, and a complete cheat sheet for synchronized • ReentrantLock • Semaphore • StampedLock



Check the link in comments!

Comment "CONCURRENCY" if this helped you